

Chapter 3

寻址方式和指令系统

Dr. Prof. Ni Li

3.1 8086寻址方式

寻址方式

- 指令: Opcode操作码 + operand操作数
 - Opcode: 操作内容
 - Operand: 操作对象
 - PUSH AX
- 指令分类
 - 无操作数, 单操作数, 双操作数
 - RET PUSH AX MOV AX, BX
 - 双操作数: 源操作数, 目的操作数

寻址方式

- Operand操作数
 - Register寄存器
 - M or I/O port (与ARM芯片不同)
 - Immediate data立即数
 - 8-bit 或16-bit 常数
- 操作数寻址效率（高->低）
 - Register: CPU(EU)
 - 立即数: 指令队列(BIU), 直接在队列里面取到
 - Memory
 - 根据偏移地址计算 20-bit 物理地址, 再访问内存
 - Offset address: effective address (EA)

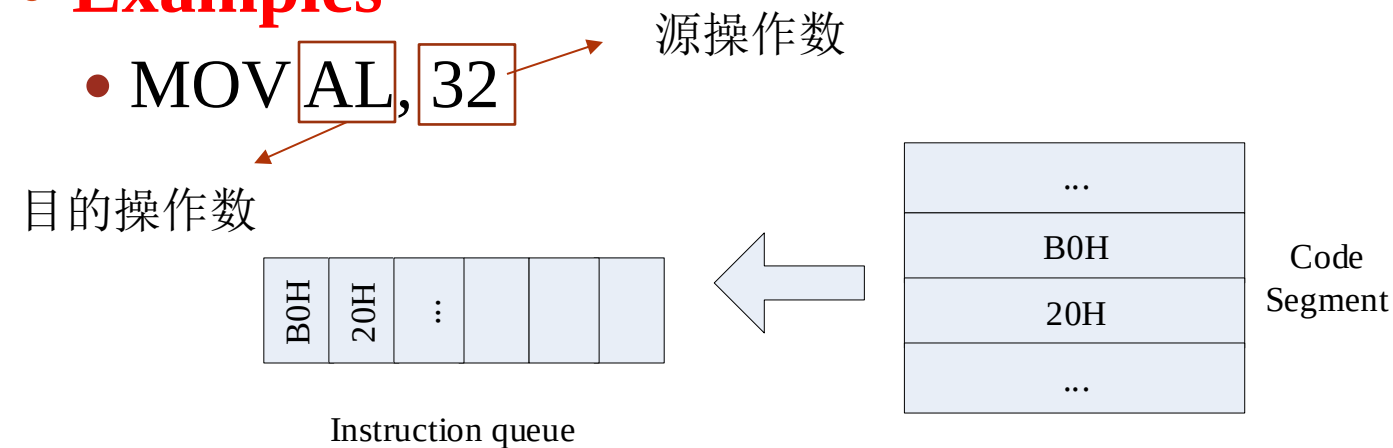
寻址方式

- **Addressing mode**寻址方式
 - 处理器访问数据的方式
- 寻址方式越多，CPU支持的指令能力越强，灵活性越好
- 寻址方式：按操作数存储位置分类
 - 寄存器和立即数寻址: 访问寄存器和立即数
 - 存储器寻址方式: accessing data in memory
 - I/O寻址方式: accessing I/O ports

立即数寻址

- Immediate addressing立即数寻址
 - 操作数是指令的一部分
 - 存储在代码段—>指令队列

- **Examples**



寄存器寻址

- Register addressing寄存器寻址
 - 操作数在寄存器中
 - 16-bit操作数
 - AX,BX,CX,DX,SI,DI,BP,SP
 - 8-bit操作数
 - AL,AH,BL,BH,CL,CH,DL,DH
- **Examples:** MOV AL, 32
- **注意**
 - 源操作数Source operand: 所有寻址方式
 - 目的操作数Destination operand: 不能是立即数寻址
 - 长度一致length consistent
 - 源操作数, 目的操作数
 - **Example:** MOV BX, AL?

→ MOV BX, AX

直接寻址

用于访问内存数据

- Direct addressing直接寻址
 - 操作数在**内存中**
 - 有效地址Effective address (EA) **直接**从指令中得到
 - 缺省段地址: **DS**
 - 操作数物理地址?
- 三种情况
 - Direct addressing直接寻址
 - Segment override prefix 段超越前缀
 - Symbol address符号地址(变量)

直接寻址

用于访问内存数据

- 直接寻址方式
 - **Example:** MOV AX, [2000H] DS=3000H [32000H]=12H
- 段超越前缀Segment override prefix
 - 堆栈段或者附加段直接寻址
 - **Example:** MOV AX, ES:[500H]
- Symbol address: 变量
 - 用符号地址代替数值地址
 - **Example:**

AREA1 DW 0867H

....

MOV AX, AREA1

MOV AX, [AREA1]

AREA1 EQU 0867H

....

MOV AX, AREA1

立即数寻址

直接寻址

- 直接寻址



寄存器间接寻址

用于访问内存数据

- Register indirect addressing mode 寄存器间接寻址
 - 操作数在memory
 - Effective address (有效地址EA) 通过寄存器给出
 - $EA = (BX, BP, SI, DI)$
- 缺省搭配
 - DS: BX, SI, DI
 - $16 \times DS + (BX, SI, DI)$
 - SS: BP
 - $16 \times SS + BP$
- **Examples (Segment override prefix)** 段超越前缀
 - MOV BX, [SI]
 - MOV AX, ES:[SI]

MOV SI, 100H

...

ADD [SI], 1;

INC SI

...

寄存器相对寻址

用于访问内存数据

- Register relative addressing 寄存器相对寻址
 - 操作数在memory
 - 有效地址EA: 在基址 (BX, BP) 或变址寄存器 (SI, DI) 基础上增加一个8位或者16位的偏移量
 - $EA = (BX, BP, SI, DI) + \text{偏移量}$
- **Example**
 - `MOV BX, [SI+4000H]   MOV BX, 4000H[SI]`

X DB ?

...

**MOV AL, X[SI];
INC SI**

基址加变址寻址

用于访问内存数据

- Based indexed addressing基址加变址寻址
 - 操作数在memory
 - 有效地址: 基址 (BX, BP) + 变址寄存器 (SI, DI)
 - $EA = (BX \text{ or } BP) + (SI \text{ or } DI)$
- 缺省搭配
 - DS—BX
 - SS—BP
 - 4种搭配组合
 - 段超越前缀: 修改缺省段基址
- **Example:** MOV AX, [BX][SI] MOV AX, [BX+SI]

相对基址加变址寻址

用于访问内存数据

- Relative based indexed addressing相对基址加变址寻址
 - 操作数在memory
 - 有效地址EA: 基址寄存器(BX or BP) + 变址寄存器(SI or DI) + 偏移量
 - $EA = (BX \text{ or } BP) + (SI \text{ or } DI) + \text{displacement}$
- 缺省搭配
 - 4种搭配组合
 - 段超越前缀: 修改缺省段基址
- **Examples**
 - `MOV AX, MASK[BX][SI]`

地址表示规则

- [Immediate data]
 - 表示有效地址(EA), eg. [2000H]
- [BX,BP,SI,DI]
 - BX 和 BP 不能同时出现在[]
 - SI 和 DI不能同时出现在[]
- []: 表示地址相加
 - 等效表示方法
- 搭配关系: BP----SS; BX/SI/DI----- DS
 - 段超越前缀: 修改缺省段基址



其他寻址方式

- IO addressing IO寻址
 - IN AL, 63H——直接 IO 寻址
 - IN AL, DX——间接 IO 寻址
- Implicit addressing 隐含寻址
 - DAA----AL (十进制调整指令)

总结

操作指令+操作对象（数据）

寻址方式	例子
立即数寻址	MOV AL, 32
寄存器寻址	MOV AL, 32
直接寻址	MOV AL, [10H]/X
寄存器间接寻址	MOV AL, [BX]
寄存器相对寻址	MOV AL, [BX+10H]
基址加变址寻址	MOV AL, [BX+SI]
相对基址加变址	MOV AL, [BX+SI+10H]

用于访问
内存数据

缺省 DS BX SI DI
SS BP

BX -- SI DI
BP – SI DI

3.2 指令机器码

机器码

- 汇编语言编程

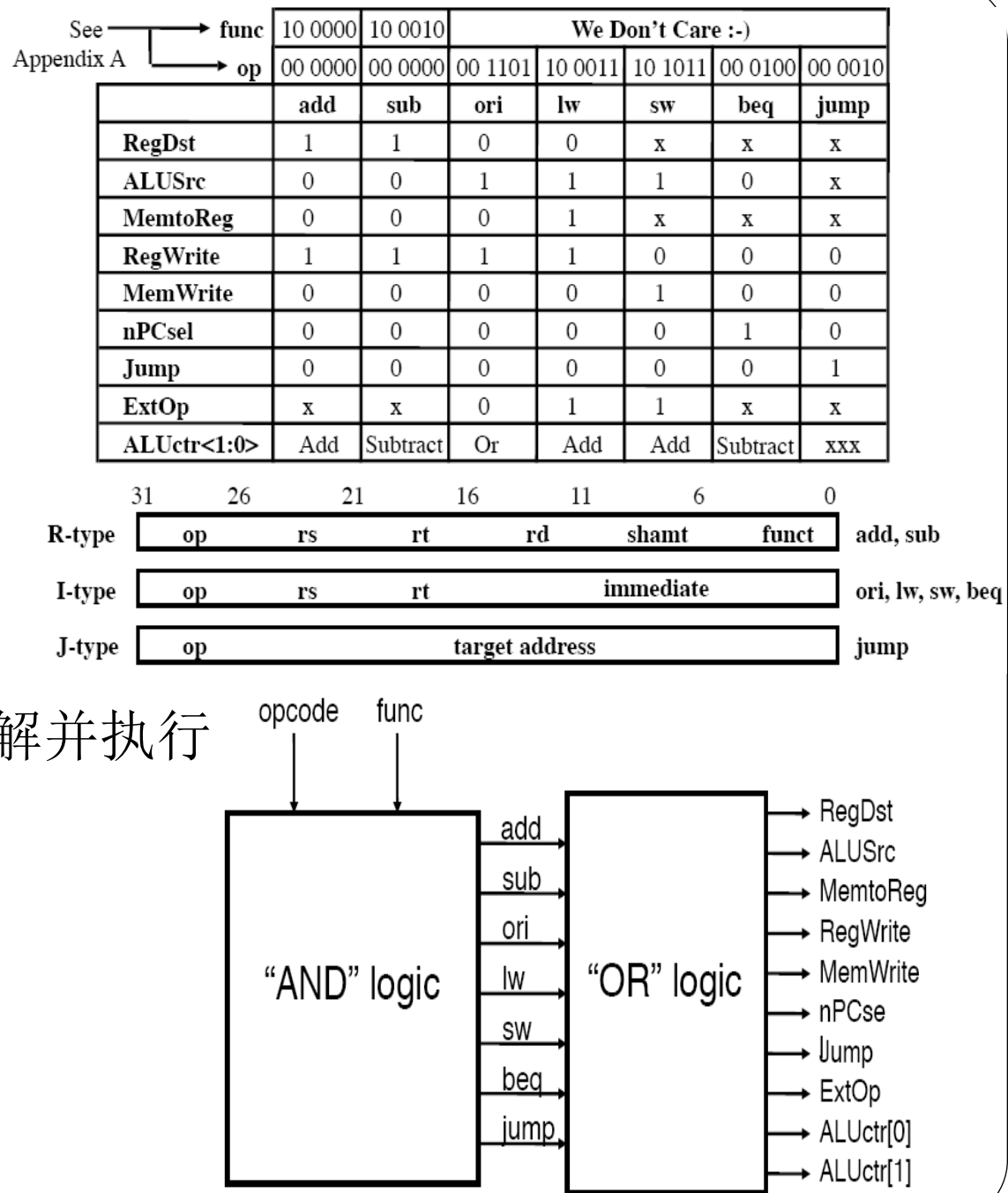
- 指令系统(指令助记符)
- 编程时不需要知道机器码

- Machine code

- 汇编指令对应的二进制码
- 汇编指令翻译成机器码才可以被计算机理解并执行
- 汇编器：汇编程序->机器码machine code

- Examples

- MOV AL, 32 → B0H 20H



汇编指令

- **指令格式Instruction format**

ADDR1: MOV AL, 32 ; move 32 to al

地址标号: 操作码+操作数 ; 注释

- 用户不能把程序存储到某个指定地址，地址标号会根据指令代码进行自动的地址分配

扩展内容

ARM Architecture & Addressing

ARM

- CISC (Complex Instruction Set Computer)
 - **Variable** instruction length可变指令长度
 - **Rich** instruction set, more **extensibility**
 - Direct access to memory locations直接访问内存
 - 复杂电路设计(decoding, execution)
- RISC(Reduced Instruction Set Computer)
 - **Fixed** instruction length固定指令长度
 - **Reduced** instruction set, 相对简单的设计
 - Load/store: 间接访问内存
 - 复杂指令由指令组合完成

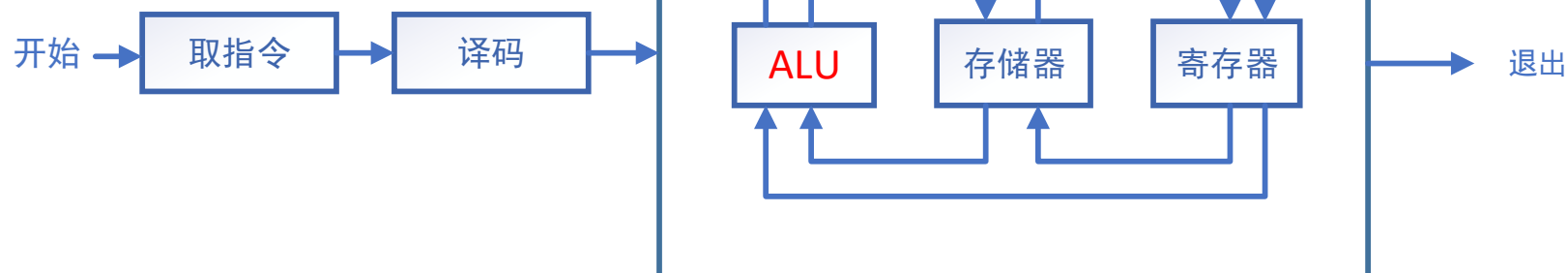
X86 Vs. ARM

- 寻址方式对比

- **MOV R0, #0xFF000**; 将立即数0xFF000装入R0寄存器
 - 立即数寻址、寄存器寻址
- **LDR R1, [R2]**; 将R2指向的存储单元的数据读出, 保存在R1中
 - 源: 寄存器间接寻址
- **LDR R2, [R3, #0X0C]**; 读取R3+0X0C地址对应存储单元的内容, 存入R2
 - 源: 寄存器相对寻址
- **STR R1, [R0]**; 把R1的值保存到R0指定的存储单元
 - 目的: 寄存器间接寻址

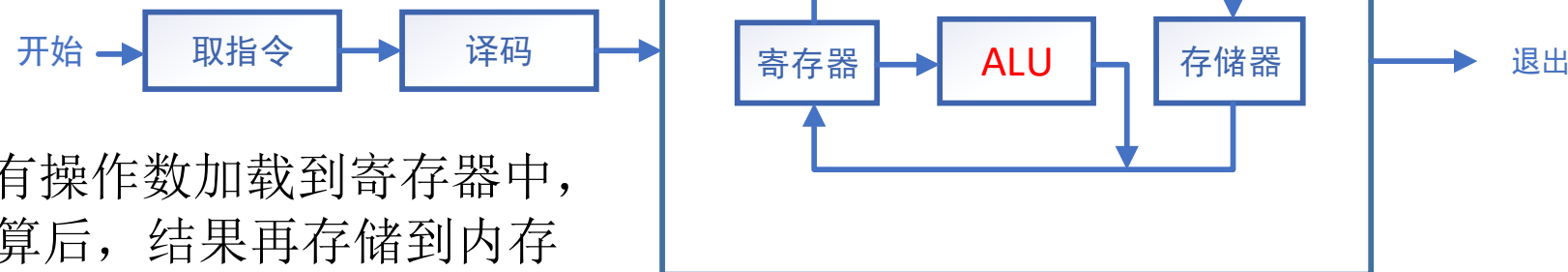
X86 Vs. ARM

- 数据传输路径



操作数可以是存储单元和寄存器，
更友好的指令调用

CISC: 寻址方式复杂



所有操作数加载到寄存器中，
完成计算后，结果再存储到内存
中。有限的指令功能带来简化的
底层硬件

RISC: Load/Store结构

X86 Vs. ARM

	CISC	RISC
指令集	Rich	Reduced, 通常<100
执行周期	执行指令时间较长 eg. 内存数据访问	执行指令时间较短
指令长度	可变长度 1-15 bytes	固定长度, 通常 4 bytes
寻址方式	多种寻址方式	相对简单
操作	对存储单元和寄存器进行算术和逻辑运算	对寄存器进行算术和逻辑运算 (Load/Store)

3.3 8086指令系统

8086 instructions

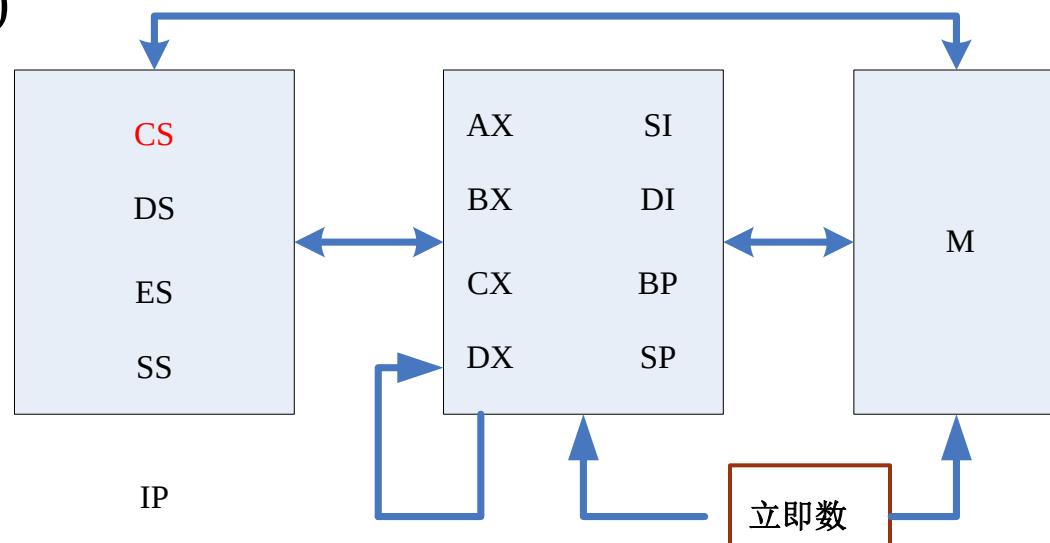
- 5 大类
 - 数据传送指令
 - 算术指令
 - 逻辑和移位指令
 - 控制转移指令
 - 串操作指令

数据传送指令

传送字节或字	
MOV	√
PUSH	√
POP	√
XCHG	√
XLAT	√
Input/output	
IN	√
OUT	√
Address transfer	
LEA	√
LDS	
LES	
Flag transfer	
LAHF	
SAHF	
PUSHF	
POPF	

数据传送指令

- MOV 指令
 - Format: MOV 目的操作数, 源操作数
 - Function: Byte/Word
- 数据传送方式(图)
 - R之间, R 和 M之间
 - 间接 (M之间, 段R之间)
 - IP CS
 - 立即数
 - 传输 word/byte



数据传送指令

● 判断对错

- MOV AX, SI
- MOV AX, BP
- MOV AX, DS
- MOV CS, AX
- MOV [BX], BX
- MOV AX, IP
- MOV DS, 12H
- MOV [BX], 12H
- MOV 100, AX

● 判断对错

- MOV AX, CS
- MOV IP, AX
- MOV [BX], 'A'
- MOV [BX], [SI]



SIM: MOV BYTE PTR [SI], 01H

数据段定义

- 数据传送指令的使用（基于数据段的定义）
- **Examples**

```
DATA SEGMENT
    X DB 12H
    Y DW 3456H
DATA ENDS
```

- SEGMENT / ENDS
- DB
- DW
- X Y 偏移地址?
- MOV AL, X 源操作数寻址方式?
- MOV AL, [0] AL?

X	12H	0
Y	56H	
	34H	

数据段定义

- Examples

DATA SEGMENT

AREA1 DB 15, 3BH

AREA2 DB 3 DUP(0)

ARRAY DW 3100H, 01A6H

STRING DB 'ABCD'

DATA ENDS

- DUP
- AREA1, AREA2, ARRAY, STRING 偏移地址?
- MOV AL, AREA1 MOV AREA2, AL
- MOV AX, [9]

AREA1	0FH	0
	3BH	
AREA2	0	9
	0	
	0	
	0	
ARRAY	0	9
	31H	
	A6H	
	01H	
STRING	'A'/41H	9
	'B'	
	'C'	
	'D'	

堆栈操作指令

- PUSH source op

- Format

- 功能

- 执行过程

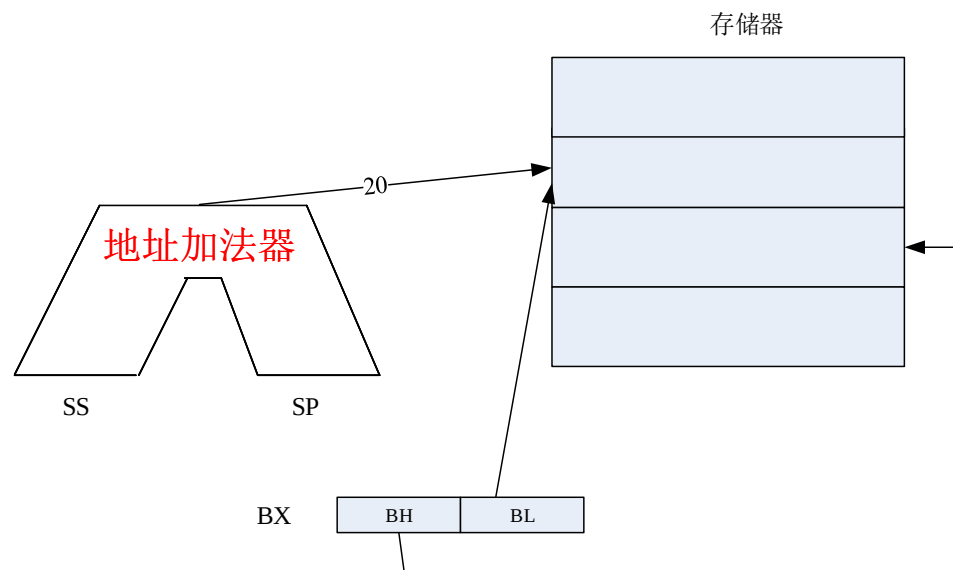
- PUSH BX

- 注意:

- 立即数不能作为源操作数直接入栈
 - 按字进行入栈操作

- Example

- 立即数如何入栈保存?



堆栈操作指令

- POP destination op

- Format

- 功能

- 执行过程

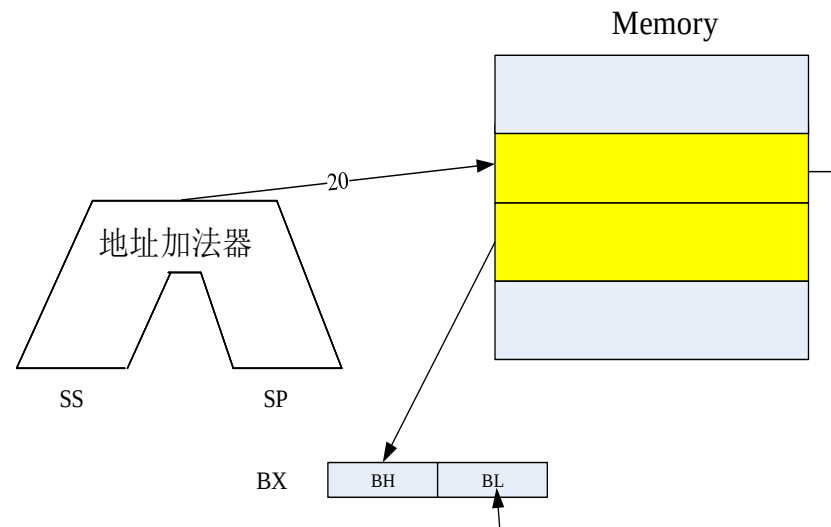
- POP BX

- 注意:

- 出栈的目的操作数不能是立即数、CS和 IP

- Example

- SS=2000H, SP=40H, BX=3120H, AX=25FEH
 - PUSH BX SP=?
 - PUSH AX SP=?
 - POP BX SP=? BX=?



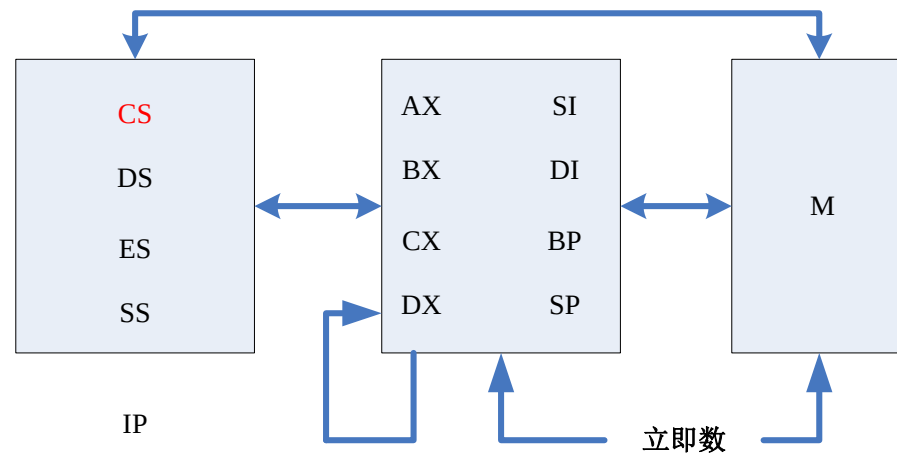
交换指令

- XCHG

- 格式: XCHG op1, op2
- 可用于数据交换
 - 寄存器之间（不能段寄存器之间）
 - 寄存器和存储单元之间
- **注意不可使用于交换:**
 - 两个段寄存器, 立即数
 - 两个存储单元

- **Example**

- AX=2000H, DS=3000H, BX=1800H, [31A00H]=1995H
- XCHG AX, [BX+200H] AX=?



取有效地址指令



- LEA: 取有效地址（偏移地址）
 - 格式: LEA 目的op, 源op
 - 功能
 - 源操作数: 存储单元 (5种寻址方式)
 - 目的操作数: 16-bit寄存器, 且不能是段寄存器
- **Examples**
 - 数据段DS=5000H, [51000H]=1234H
 - LEA BX, [SI], BX=? MOV BX, [SI], BX=?
 - LEA BX, TABLE ↔ MOV BX, OFFSET TABLE
 - OFFSET后只能接变量

XLAT查表指令

- XLAT

- 格式

- XLAT

- XLAT 表名

- 功能

- 根据偏移量查表

- 先在内存中建立表(最多256 bytes)

- 表头偏移地址->BX

- 想查找元素相对偏移 ->AL

- Result->AL

- Examples

- ASCII码

表头EA地址:BX

X0	0 偏移: AL
X1	1
X2	2
X3	3
X4	4
.....	

DATA SEGMENT

TABLE1 DB 30H, 31H, 32H,....

DATA ENDS

.....

MOV AL, 5

MOV BX,OFFSET TABLE1

XLAT; **AL中为35H**

Input/output指令

- 数据传输：I/O 端口与累加器之间(AL, AX)
 - IN OUT
- IN
 - Format (直接/间接寻址)
 - Function
 - Examples (判断对错)
 - IN AX, 80H / MOV DX, 80H IN AX, DX
 - IN AX, 100H
 - IN BL, 20H
 - IN AL, [20H]
 - MOV DX, 0FF4H IN AL, DX

Input/output指令

- OUT
 - Format (直接/间接寻址)
 - eg: OUT 80H, AL / OUT DX, AX
 - Function
 - Examples

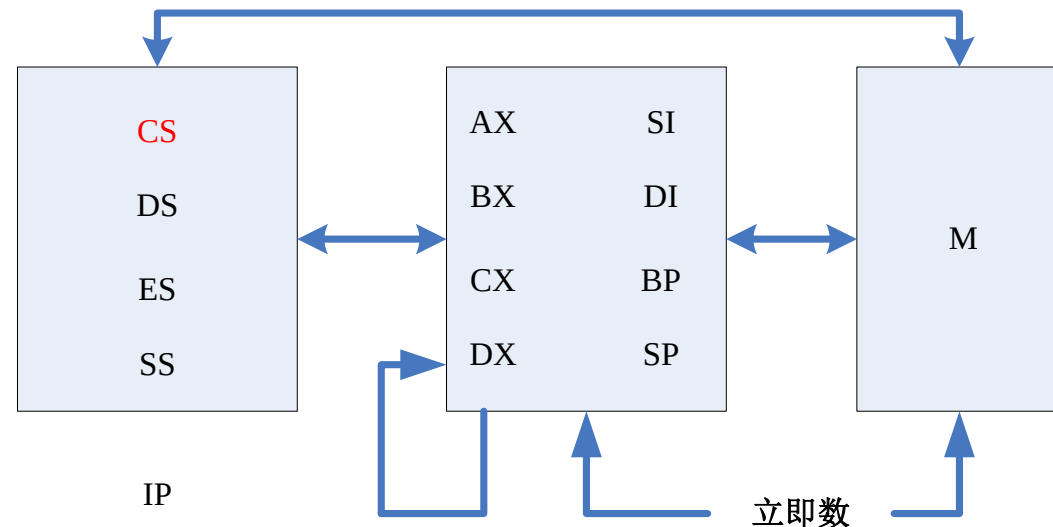


算术运算指令 Arithmetic instruction

- ALU
- FR: ALU运算结果
 - 9 位有效
 - CF、PF、AF、OF、SF、ZF
- 加法和减法
 - Unsigned / signed binary integer
 - Unsigned packed/ unpacked decimal integer
 - 浮点数计算: 协处理器/软件处理

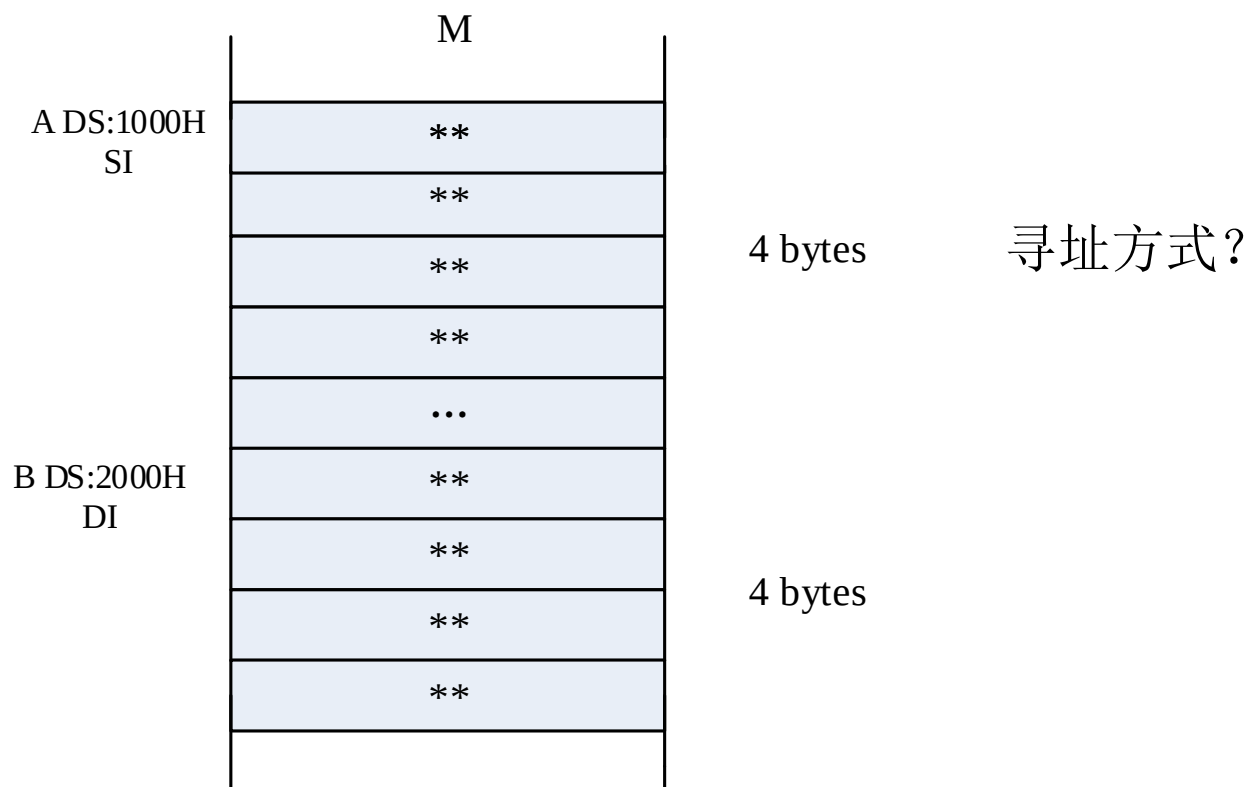
加法Addition instruction

- ADD: 加法
 - Source Op + destination Op $\rightarrow$ destination Op
 - 影响**FR**的6个标志位
 - **Examples:**
 - ADD AL, BL
 - ADD AL, [SI]
 - **ADD [BX], [SI]**
 - **ADD AL, BX**
 - **ADD 100, AX**



加法

- ADC: addition with carry 带借位加法
 - Source Op + destination Op + CF $\rightarrow$ destination Op
 - 影响 **FR** 的 6 个标志位
 - **Examples:** 取 CF? 四字节整数加法?



加法指令

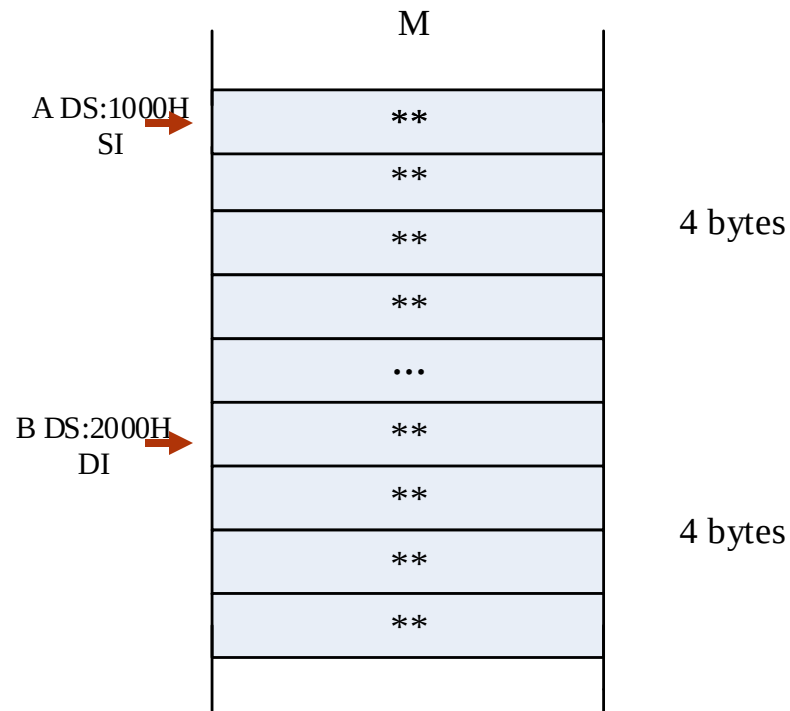
MOV SI, 1000H

MOV DI, 2000H

MOV AX, [SI]

ADD AX, [DI]; 低16位相加

MOV [SI], AX; 保存低位结果



MOV AX, [SI+2]; 此指令不影响CF标志

ADC AX, [DI+2]; 高16位带CF相加

MOV [SI+2], AX

MOV AL, 0

ADC AL, 0; 取高16位相加后的CF

MOV [SI+4], AL; 将进位放在存储单元中

寄存器间接寻址:
程序可扩展性?
当EA变化时?

加法指令

- INC: increment 自加指令
 - 不影响CF
 - 单操作数: 当操作数是存储单元时, 要指明是对字节还是字进行操作
 - **Examples**
 - INC BL
 - INC CX
 - INC **BYTE PTR**[BX]
 - INC **WORD PTR**[BX]

加法指令

引入BCD码表示十进制：
简化计算（二进制加法器）

- DAA: 十进制调整指令
 - 把两个压缩型BCD码的和调整成正确的BCD码
 - **AL** (implicit addressing隐含寻址)
- 调整过程
 - 若AL低4位大于9或AF=1, 低4位+6
 - 如果AL中高4位大于9或CF=1, 高4位+6, 且CF=1; 否则 CF=0
- **Examples**
 - 88 + 49
 - ADD **AL**, BL; 1000 1000+0100 1001 = 1101 0001, AF=1
 - DAA ; 1101 0001+**0000 0110** =1101 0111, 低4位+ 6
 - ; 1101 0111+**0110 0000** = 0011 0111, 高4位+6H
 - 37 BCD, **CF=1**

加法指令

- AAA: ASCII 加法调整
 - 把两个非压缩型BCD码的和调整成正确的BCD码
- 调整过程
 - 若AL低4位大于9或AF=1, 低4位+6, 并把AL高4位清零; AF=1, CF=1, 且 $AH \leftarrow AH+1$
 - 否则, 把AL高4位清零
- Examples
 - 9 BCD + 5 BCD
 - ADD **AL**, BL ; 0000 1001+0000 0101 = 0000 1110
 - AAA ; 0000 1110+**0000 0110** = 0001 0100 , 低4位+6
; 0001 0100 $\wedge$ 0000 1111 = 0000 0100, 高4位清零
CF=1, AF=1, AH=1

减法Subtraction instruction

- SUB: 减法

- Destination Op = destination Op - source Op

- **Examples**

- SUB AX, BX
- SUB DX, 1850H
- SUB BL, [BX]

影响**FR**

- SBB: 带借位的减法

- Destination Op = destination Op - source Op - CF

- **Examples**

- SBB AL, CL; AL ← AL - CL - CF

减法

- DEC: decrement
 - 不影响**CF**
 - **Examples**
 - DEC BX
 - DEC **WORD PTR**[BP]

减法

- CMP: 比较, 不影响目的操作数
 - Destination Op – source Op -->影响FR
 - **Examples: CMP AL, 30H (ASCII码比较)**
- 十进制调整指令
 - DAS (packed)
 - AAS (unpacked)
 - AL

练习签到



逻辑操作指令

- NOT: 取反
 - 目的操作数不能是立即数
 - 对FR无影响
 - **Examples**
 - NOT AL
 - NOT AX
 - NOT **BYTE PTR**[BX]
- AND/OR
 - Format: AND/OR destination op, source op
 - **Example: AX=3538H (“5” “8” ASCII) → AH, AL非压缩型 BCD**
 - 取不同二进制位: D7 D2 D1 D0
 - 影响FR

逻辑操作指令

- TEST

- 进行与操作, 但是**不改变操作数**

- 影响**FR**

- **Examples**

- TEST AL, 80H ; 判断正负

- JNZ NEXT ; 结果不为0则跳转（符号位为1，负数）

- TEST AL, 40H

- TEST AL, 1 ; 判断奇偶

- JZ NEXT1 ; 结果为0则跳转（末尾位为0，偶数）

逻辑操作指令

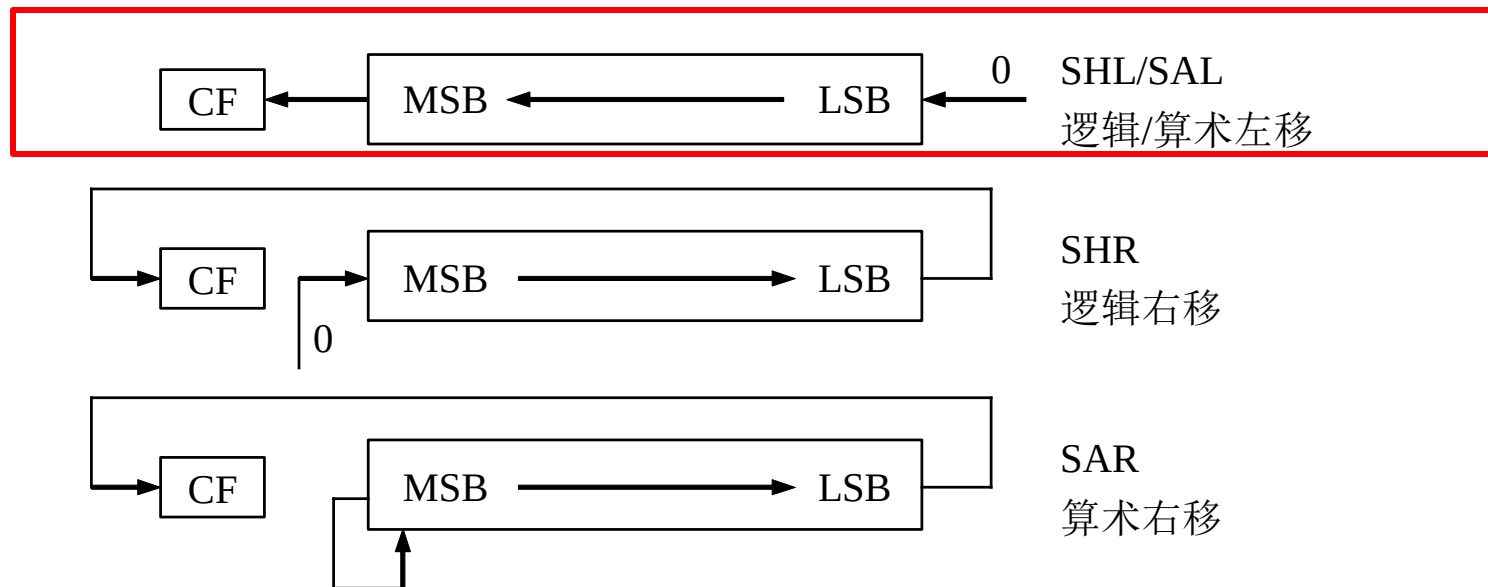
- XOR (相同->0, 不同->1)
 - 清零操作
 - 与0异或保持不变/ 与1异或取反
 - XOR AL, 0FH
 - XOR BX, 0C000H
 - 简单加密方法

```
MOV AL, 'A'; 41H 01000001B
XOR AL, 55H;
XOR AL, 55H;
```

逻辑操作指令

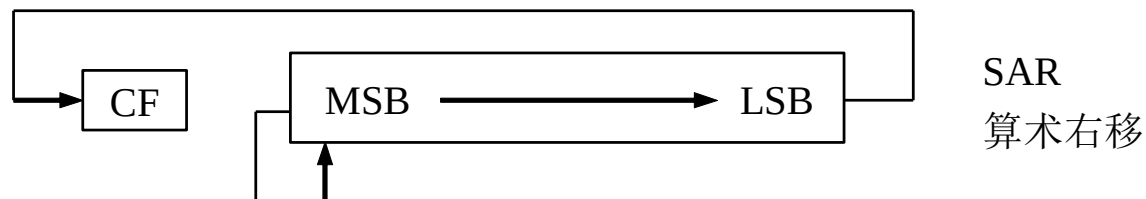
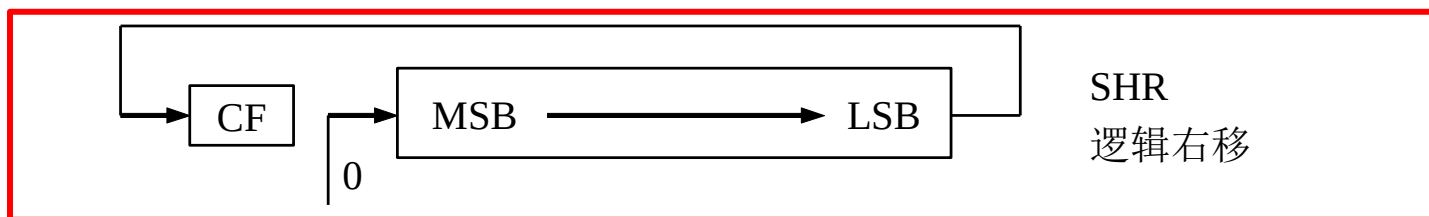
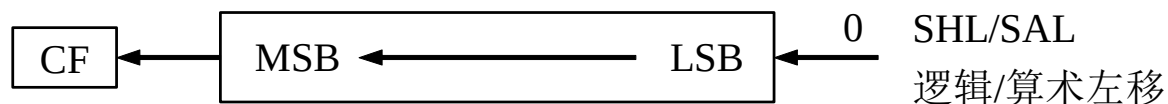
- 双操作数
 - 源操作数：R, M 或 立即数;
 - 目的操作数：不能为立即数 AND 12H, AL (错误)
 - 两个操作数不能都为存储单元 AND [SI], [BX] (错误)
 - **CF 和 OF均清零**
 - OR AX, AX / CLC
 - **ZF, SF 和 PF受结果影响**
- 单操作数(NOT)
 - 目的操作数：不能为立即数 NOT 12H (错误)
 - 单操作数: 当操作数是存储单元时，要指明是对字节还是字进行操作
NOT BYTE PTR [BX]
 - **不影响FR: 同INC DEC**

移位指令



- SHL/SAL op, 1/CL
 - Format: 如果移位次数 > 1, 移位次数 -> CL
 - LSB (Least Significant Bit) 为 0, MSB (Most Significant Bit) 移至 CF
 - **Examples: 左移一次相当于乘以2 有无符号数适用**
 - SAL AH, 1
 - AH=06H, FFH(-1补码), 40H (64)

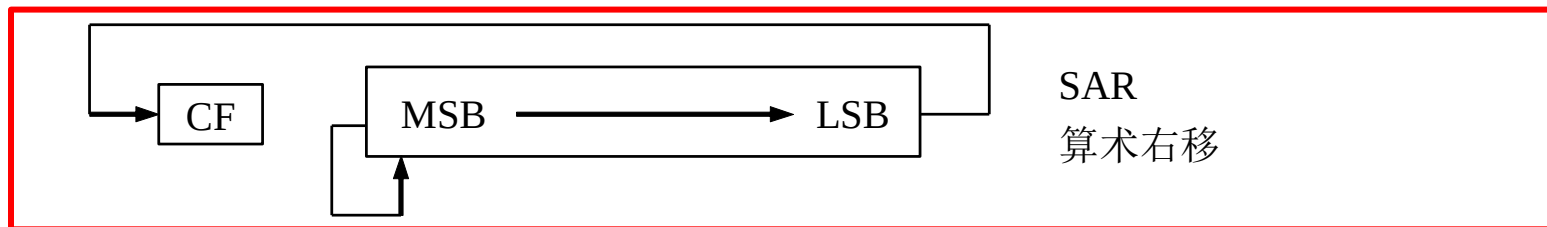
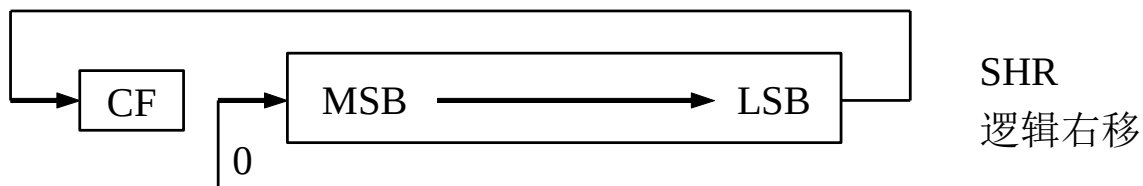
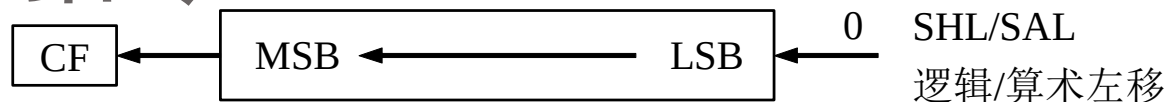
移位指令



- SHR

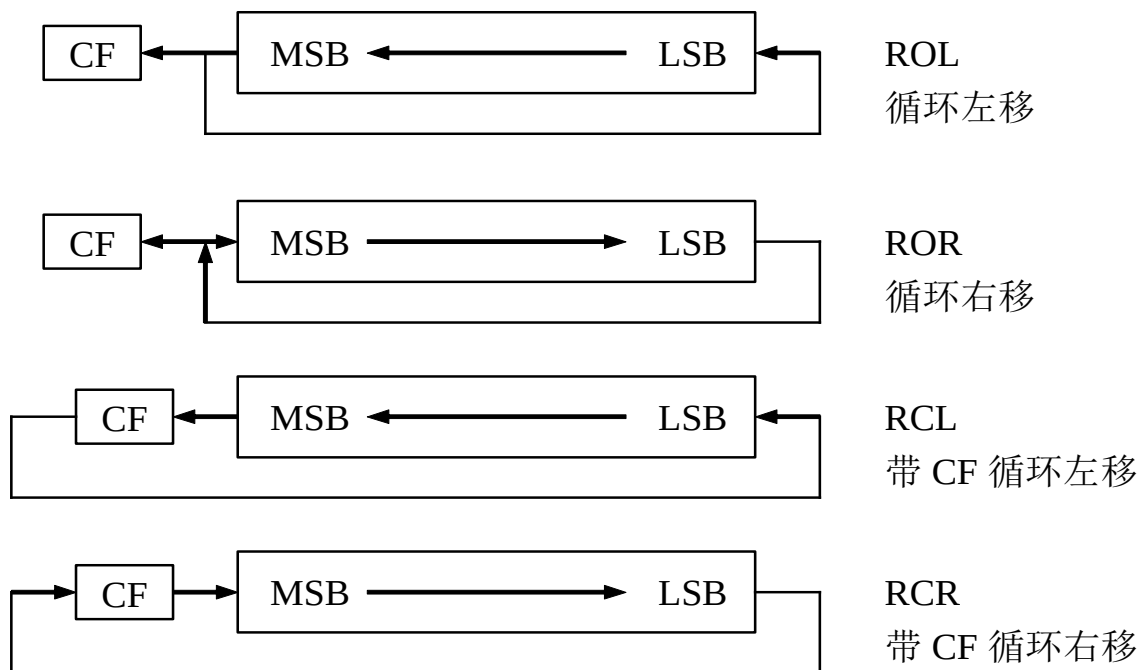
- LSB 移至CF, MSB 为 0
- 无符号数: 右移一次相当于除以2(舍弃余数)
- **Examples**
 - SHR AL, CL; CL=3
 - AL=10000101B (133) --> 16

移位指令



- SAR
 - LSB 移至 CF, MSB 保持不变
 - 有符号数: 右移一次相当于除以2 (补码表示)
 - **Examples**
 - SAR AL, CL; CL=3
 - AL=10000000B (-128)

循环指令



- ROL/ROR: 不带CF循环, 如果移位次数 > 1, 移位次数->CL
- RCL/RCR: 带CF循环, CL
- **Examples**
 - ROL BX, CL
 - ROR WORD PTR [SI], 1

Example

- $AL = \underline{10101001}B = A9H$ ASCII码 'A' $\rightarrow$ AH '9' $\rightarrow$ AL

MOV AL, 0A9H; 加上0以区分变量名和数据

MOV AH, AL; 在AH中备份

AND AL, ____; 先取AL低4位, 转换 '9'

ADD AL, 30H; 变成9的ASCII码

MOV CL, ____; 移位位数

____ AH, CL; 逻辑右移指令

ADD AH, ____; 变成A的ASCII码

控制转移指令

无条件转移和过程调用指令	
JMP	无条件转移
CALL	过程调用
RET	过程返回
条件转移	
JZ/JE 等 10 条指令	直接标志转移
JA/JNBE 等 8 条指令	间接标志转移
条件循环控制	
LOOP	CX≠0 则循环
LOOPE/LOOPZ	CX≠0 和 ZF=1 则循环
LOOPNE/LOOPNZ	CX≠0 和 ZF=0 则循环
JCXZ	CX=0 则转移
中断	
INT	中断
INTO	溢出中断
IRET	中断返回

无条件转移指令 ---JMP

- 根据转移距离分类:
 - **段内转移**Intrasegment jump (short/near jump)
 - 在当前段跳转
 - 改变IP, CS不变
 - 段间转移Intersegment jump (Far jump)
 - 跳转到其他段
 - IP, CS都改变
- 根据寻址方式分类: **直接** 和 间接

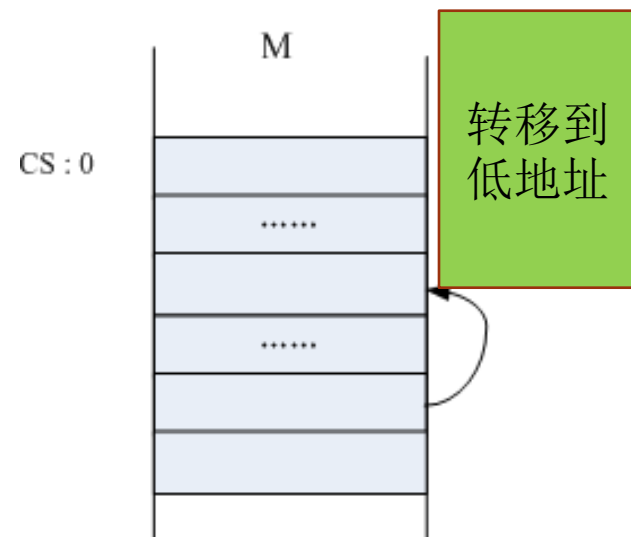
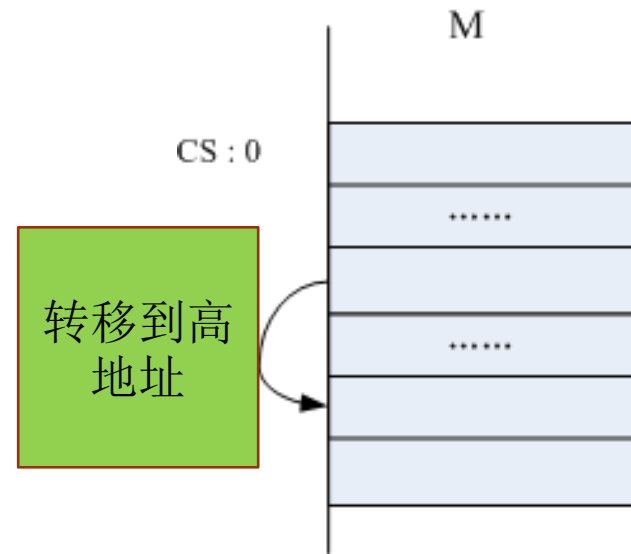
无条件转移指令 ---JMP

- 段内短转移
 - **Format:** JMP SHORT Address-label
 - 2字节指令
 - 转移范围: Jump **下一条指令** [-128,127]
- 偏移量: 8 位补码
 - 跳转到低地址, 偏移量为负数的补码
 - 跳转到高地址, 偏移量为正数的补码

段内短转移



$IP \leftarrow IP + 8 \text{ 位偏移量}$



无条件转移指令 ---JMP

段内短转移

- **Examples**

- 偏移量计算?

1234H: 90 NEXT:

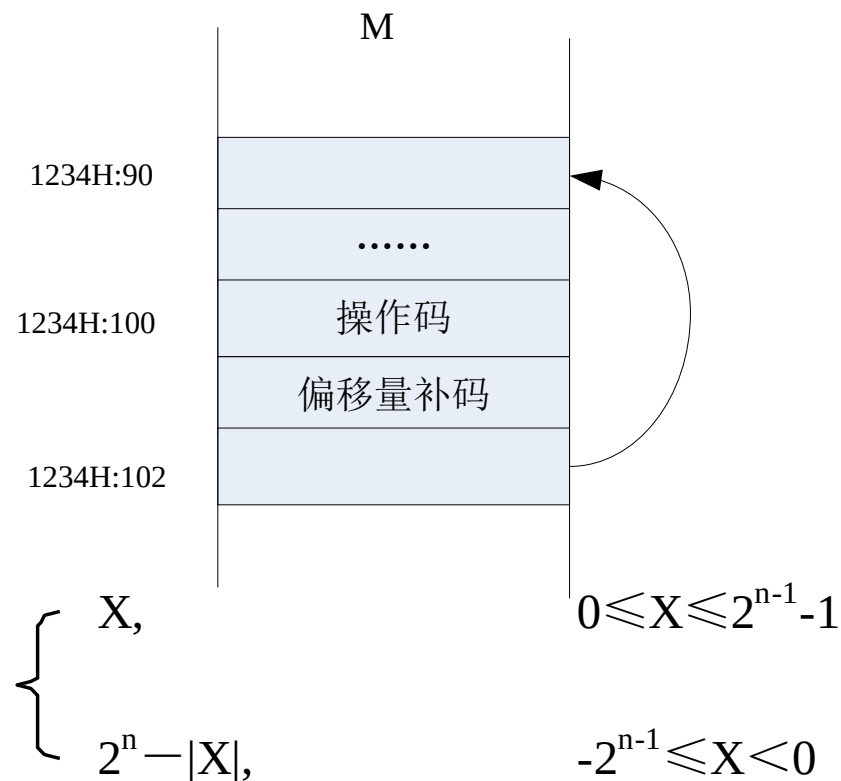
.....

1234H: 100 JMP SHORT NEXT



EB	DISP_L
----	--------

$IP \leftarrow IP + 8\text{-bit 偏移量}$



$$[X]_{\text{补}} = \begin{cases} X, & 0 \leq X \leq 2^{n-1} - 1 \\ 2^n - |X|, & -2^{n-1} \leq X < 0 \end{cases}$$

-12的补码: 100H-C=F4H

无条件转移指令 ---JMP

- 段间近转移
 - **Format:** JMP NEAR PTR Address-label
 - **Near jump:** 转移范围为Jump **下一条指令** [-32768, +32767]
 - 3字节指令
 - SHORT 和 NEAR PTR编程时可省略

段内近转移

E9	DISP_L	DISP_H
----	--------	--------

$IP \leftarrow IP + 16$ 位偏移

段内短转移

EB	DISP_L
----	--------

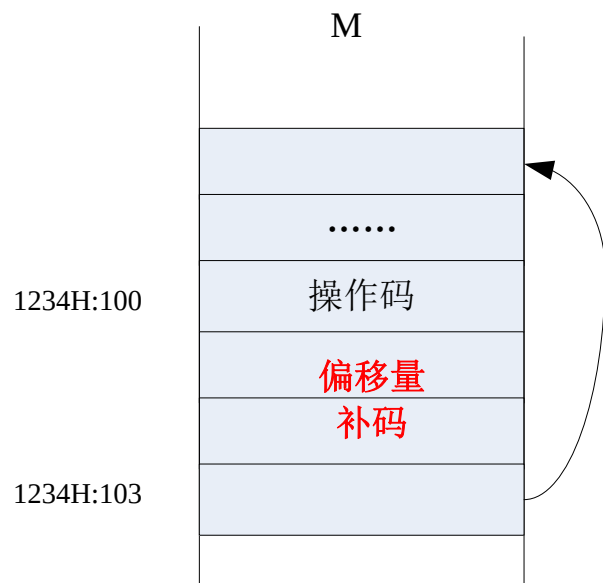
$IP \leftarrow IP + 8$ 位偏移

无条件转移指令 ---JMP

- **Examples**

- 1234H:100 JMP NEAR PTR NEXT

- **IP?**



段内近转移

E9	DISP_L	DISP_H
----	--------	--------

$IP \leftarrow IP + 16$ 位偏移

段内短转移

EB	DISP_L
----	--------

$IP \leftarrow IP + 8$ 位偏移

无条件转移 ---JMP

- 总结

类型	方式	寻址目标	指令举例
段内转移	直接	立即短转移（8 位）	JMP SHORT PROG_S
		立即近转移（16 位）	JMP NEAR PTR PROG_N
	间接	寄存器（16 位）	JMP BX
		存储器（16 位）	JMP WORD PTR 5[BX]
段间转移	直接	立即转移（32 位）	JMP FAR PTR PROG_F
	间接	存储器（32 位）	JMP DWORD PTR [DI]

CALL and RET instructions

- 子程序
 - 内存中的一组指令->实现特定功能
 - 有可能被主程序调用
 - 以**PROC**开头, 以**RET ENDP**介绍
- **CALL**: 执行子程序
- **RET**: 返回主程序中调用CALL指令的下一条指令
- Format
 - CALL 子程序名称
 - RET

CALL and RET instructions

- **Examples**

```
MAIN PROC
```

```
... ..
```

```
MOV AX, 3538H
```

```
CALL P1
```

```
ADD AX, CX
```

```
... ..
```

```
MAIN ENDP
```

```
P1 PROC NEAR
```

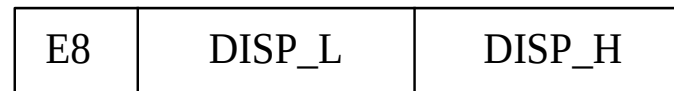
```
AND AX, 0F0FH
```

```
RET
```

```
P1 ENDP
```

CALL and RET instructions

- 近调用和远调用
 - 近调用: 调用同一个代码段的程序
 - 远调用: 调用不同代码段的程序
- 段内直接调用(近调用)
 - 3字节指令, 2字节表示偏移量



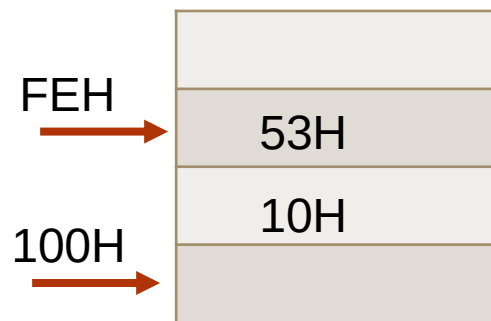
$IP \leftarrow IP + 16 \text{ 位偏移量}$

CALL and RET instructions

- CALL执行过程
 - 返回地址压入堆栈: **CALL**指令之后那条指令的地址
 - 近调用: IP
 - 远调用: CS IP
 - 转到子程序入口地址执行相应程序: 根据2个字节偏移量
- RET执行过程
 - 返回地址出栈
 - 近调用: IP
 - 远调用: IP CS
 - 返回主程序

CALL and RET instructions

- **Examples:** CALL PROG_N
 - 调用前: CS=2000H IP=1050H SS=5000H SP=0100H
 - CALL指令中2字节偏移量: 1234H
 - PROG_N 程序代码地址? 返回地址? 堆栈变化?
 - 过程: 入栈, 跳转, 出栈, 返回



条件转移指令

- 根据当前标志寄存器中标志位的状态来决定是否转移
 - **2-byte instruction (machine code)**
 - **短转移**:允许跳转的距离(-128 - +127)
- Conditional transfer instructions 条件转移指令
 - Direct flag transfer instructions 直接条件转移
 - Indirect flag transfer instructions 间接条件转移
 - Iteration control instructions 循环控制

条件转移指令

- 直接标志条件

指令助记符	测试条件	指令功能
JC	CF=1	有进位 转移
JNC	CF=0	无进位 转移
JZ/JE	ZF=1	结果为 0/相等 转移
JNZ/JNE	ZF=0	不为 0/不相等 转移
JS	SF=1	符号为负 转移
JNS	SF=0	符号为正 转移
JO	OF=1	溢出 转移
JNO	OF=0	无溢出 转移
JP/JPE	PF=1	奇偶位为 1/为偶 转移
JNP/JPO	PF=0	奇偶位为 0/为奇 转移

条件转移指令

- **Example**

```
    ADD  AL, BL
    JC   NEXT
    MOV  AH, 0
    JMP  EXIT
NEXT: MOV  AH, 1
EXIT: ...
```

- 当超出-128~+127范围?
 - **JMP**

```
    MOV  BX, 10
    ....
NEXT:
    ....
    DEC  BX
    JNZ NEXT
    ↓ 代码调整
    MOV  BX, 10
    ....
NEXT:
    ....
    DEC  BX
    JZ NEXT1
    JMP NEXT
NEXT1:.....
```

扫码练习



条件转移指令

- **J***

- 放在比较指令**CMP**后，间接标志转移

类别	指令助记符	测试条件	指令功能
无符号数 比较测试	JA/JNBE	CF $\vee$ ZF=0	高于/不低于等于 转移
	JAE/JNB	CF=0	高等于于/不低于 转移
	JB/JNAE	CF=1	低于/不高于等于 转移
	JBE/JNA	CF $\vee$ ZF=1	低于等于/不高于 转移
带符号数 比较测试	JG/JNLE	(SF $\vee$ OF) $\vee$ ZF=0	大于/不小于等于 转移
	JGE/JNL	SF $\vee$ OF=0	大于等于/不小于 转移
	JL/JNGE	SF $\vee$ OF=1	小于/不大于等于 转移
	JLE/JNG	(SF $\vee$ OF) $\vee$ ZF=1	小于等于/不大于 转移

A—Above, B—Below, E—Equal, G—Greater than, N—not , L—Less than

条件转移指令

- **Examples**

MOV AL, X

CMP AL, X+1

JB NEXT;

XCHG AL, X+1

MOV X, AL

NEXT:

X

0DH
FFH

JB->JL ?

循环控制指令

- Iteration control instructions 循环控制
 - 按照一定次数执行一系列指令; 执行次数→**CX**
 - 跳转目的地址范围: 循环指令下一条指令地址**-128~+127** bytes

- LOOP

- 等效指令

MOV CX, 10

- **Examples:** 延时程序

DELAY: LOOP DELAY

- LOOPE/LOOPZ

- ZF=1 且 CX≠0

- LOOPNE/LOOPNZ

- ZF=0 且 CX≠0

MOV CX, 10

DELAY: DEC CX

JNZ DELAY

串操作指令

- String
 - 一系列存放在存储器中的字或字节数据
 - 串长度可达64KB
 - 对**存储单元**的数据操作

指令名称	字节/字操作	字节操作	字操作
串传送	MOVS 目的串，源串	MOVSB	MOVSW
串比较	CMPS 目的串，源串	CMPSB	CMPSW
串扫描	SCAS 目的串	SCASB	SCASW
串装入	LODS 源串	LODSB	LODSW
串存储	STOS 目的串	STOSB	STOSW

串操作指令

- MOVSB/MOVSW
 - $[DS:SI] \rightarrow [ES:DI]$ $SI = SI + (-)1(2)$ $DI = DI + (-)1(2)$
 - **Example: ADDR1 $\rightarrow$ ADDR2 100 bytes**
- 隐含规则
 - 源串: DS SI
 - 目的串: ES DI
 - SI, DI: 自动修改
 - DF: 方向控制
 - DF=0 递增方向, 每执行一次字节/字串操作, SI和DI都增1/2 CLD
 - DF=1 递减方向, 每执行一次字节/字串操作, SI和DI都减1/2 STD
- 与REP配合使用
 - 重复次数 $\rightarrow$ CX
 - REP: 如果 $CX \neq 0$ 则重复

串操作指令

MOV AX, SEG ADDR1; 取出ADDR1的段地址

MOV DS, AX; **立即数不能直接送入段寄存器中**

LEA SI, ADDR1; 取ADDR1有效地址

MOV AX, SEG ADDR2; 取出ADDR2的段地址

MOV ES, AX

LEA DI, ADDR2; 取ADDR2有效地址

CLD; DF标志清零, SI、DI自动加

MOVS B; 执行100次, 或者MOVSW执行50次

.....

MOVS B; 执行100次, 或者MOVSW执行50次

可用以下语句代替:

MOV CX, 100

REP MOVS B

串操作指令

- LODSB/ LODSW
 - $[DS:SI] \rightarrow AL, AX \quad SI = SI + (-)1(2)$
- STOSB / STOSW
 - $AL, AX \rightarrow [ES:DI] \quad DI = DI + (-)1(2)$
 - **Examples: 内存初始化**

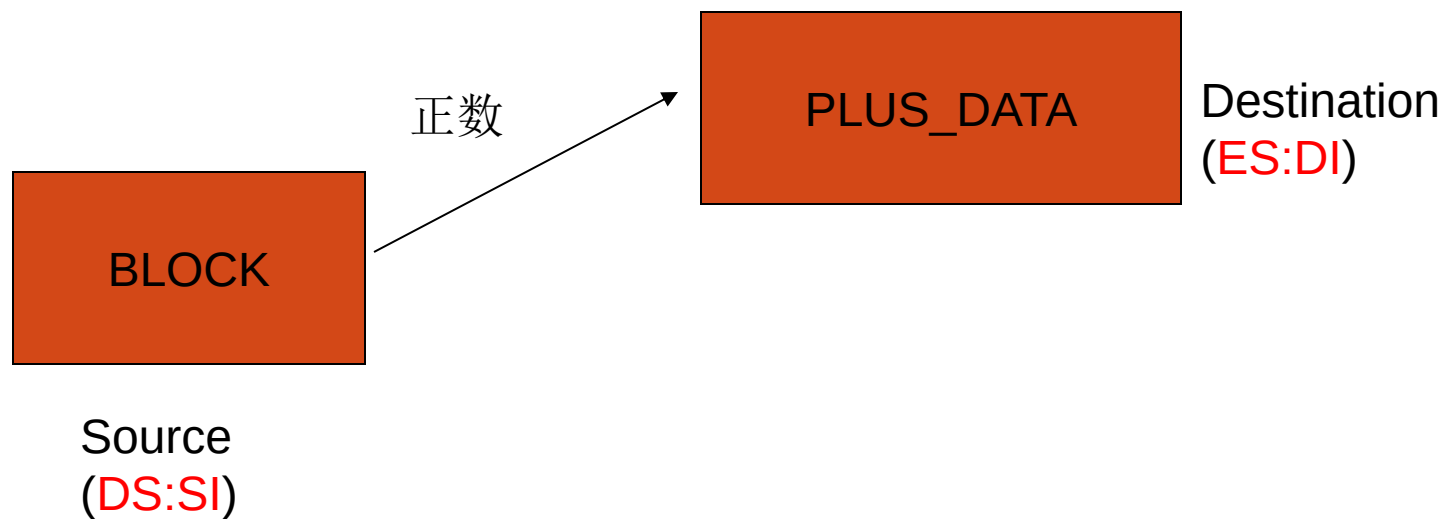
MOV AL,0 ; 假设ES, DI已指向某内存单元

MOV CX, 100

REP STOSB

Examples

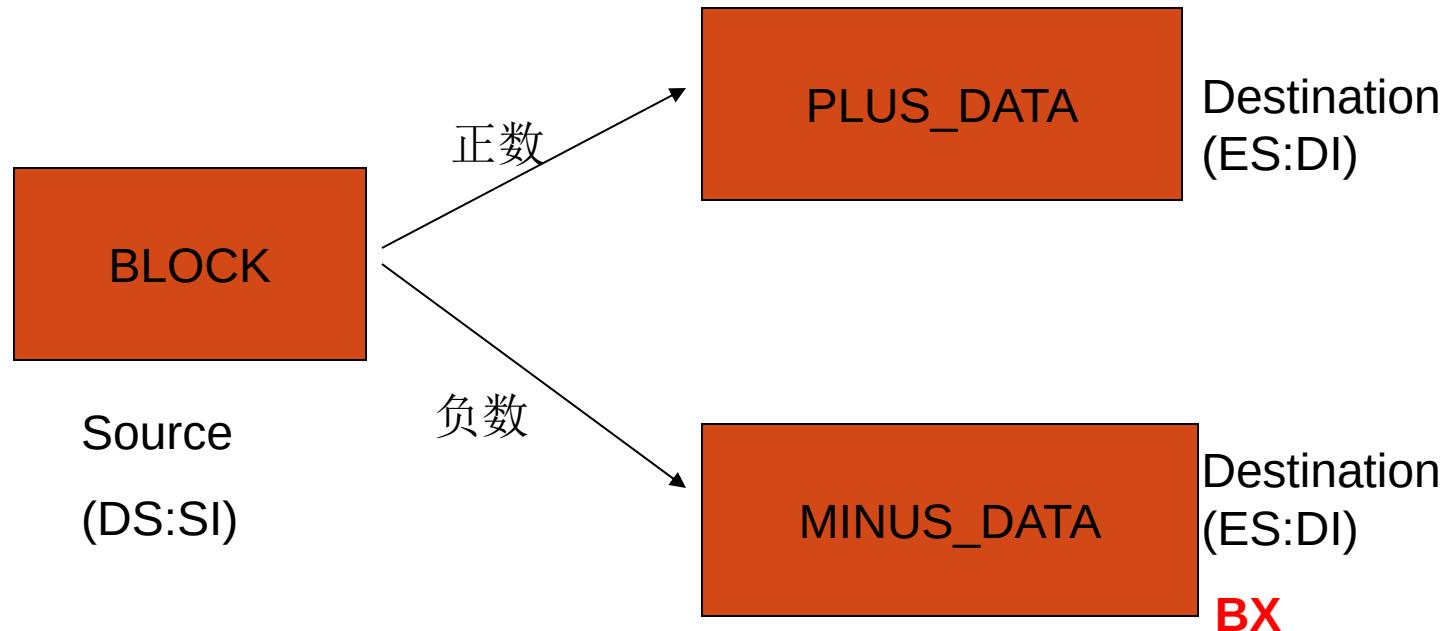
- 串的起始地址为BLOCK，有COUNT个字节。把正数放到起始地址为PLUS_DATA的串中
- DS , ES已分别存放 BLOCK 和 PLUS_DATA的段地址



```
START: MOV SI, OFFSET BLOCK;源串
        MOV DI, OFFSET PLUS_DATA; 目的串
        MOV CX, COUNT; 循环次数
        CLD; SI、DI自动加
GOON:  LODSB; [DS:SI]->AL
        TEST AL, 80H
        JNZ AGAIN; 符号位为1, 负数
        STOSB; AL-> [ES:DI], 正数放到PLUS_DATA
AGAIN: DEC CX
        JNZ GOON
```

Examples

- 串的起始地址为BLOCK，有COUNT个字节。把正数放起始地址为PLUS_DATA的串中，负数放起始地址为MINUS_DATA的串中
- DS 中为BLOCK段地址，ES中为PLUS_DATA 和 MINUS_DATA的段地址



```
START: MOV    SI, OFFSET BLOCK; 源串
        MOV    DI, OFFSET PLUS_DATA; 目的串
        MOV    BX, OFFSET MINUS_DATA
        MOV    CX, COUNT
        CLD
GOON:   LODSB
        TEST   AL, 80H
        JNZ    MINUS; 负数跳转到MINUS
        STOSB
        JMP    AGAIN
MINUS:  XCHG   BX, DI
        STOSB; 负数到MINUS_DATA
        XCHG   BX, DI
AGAIN:  DEC    CX
        JNZ    GOON
```

通用寄存器的特殊用法

寄存器名	特殊用途
AX, AL	在输入输出指令中作数据寄存器
AL	在十进制运算指令中作累加器 在 XLAT 指令中作累加器
BX	在间接寻址中作基址寄存器 在 XLAT 指令中作基址寄存器
CX	在串操作指令和 LOOP 指令中作计数器
CL	在移位/循环移位指令中作移位次数寄存器
DX	在间接寻址的输入输出指令中作地址寄存器
SI	在字符串运算指令中作源变址寄存器 在间接寻址中作变址寄存器
DI	在字符串运算指令中作目标变址寄存器 在间接寻址中作变址寄存器
BP	在间接寻址中作基址指针
SP	在堆栈操作中作堆栈指针

Review of chapter 3

- 指令格式: [地址标号:] 操作码 操作数[; 注释]
- 寻址方式
 - 立即数
 - 寄存器
 - 存储器5种: 直接、寄存器间接、寄存器相对、基址加变址、相对基址加变址
- Instruction system
 - 数据传送(MOV, PUSH/POP, XCHG, XLAT, IN/OUT, LEA), 算术, 逻辑和移位, 控制转移, 串操作

- 练习



作业

- P109 3-1 3-2 3-3 3-6 3-7 3-10

习 题

1/ 分别说明下列指令的源操作数和目的操作数各采用什么寻址方式。

- | | |
|------------------------|------------------------|
| (1) MOV AX, 2408H | (2) MOV CL, 0FFH |
| (3) MOV BX, [SI] | (4) MOV 5[BX], BL |
| (5) MOV [BP+100H], AX | (6) MOV [BX+DI], '\$' |
| (7) MOV DX, ES:[BX+SI] | (8) MOV VAL[BX+DI], DX |
| (9) IN AL, 05H | (10) MOV DS, AX |

2/ 已知: DS=1000H, BX=0200H, SI=02H, 内存 10200H~10205H 单元的内容分别为 10H, 2AH, 3CH, 46H, 59H, 6BH。下列每条指令执行完后 AX 寄存器的内容各是什么?

- | | |
|---------------------|----------------------|
| (1) MOV AX, 0200H | (2) MOV AX, [200H] |
| (3) MOV AX, BX | (4) MOV AX, 3[BX] |
| (5) MOV AX, [BX+SI] | (6) MOV AX, 2[BX+SI] |

3/ 设 DS=1000H, ES=2000H, SS=3500H, SI=00A0H, DI=0024H, BX=0100H, BP=0200H, 数据段中变量名为 VAL 的偏移地址值为 0030H, 试说明下列源操作数字段的寻址方式是什么? 物理地址值是多少?

- | | |
|------------------------|------------------------|
| (1) MOV AX, [100H] | (2) MOV AX, VAL |
| (3) MOV AX, [BX] | (4) MOV AX, ES:[BX] |
| (5) MOV AX, [SI] | (6) MOV AX, [BX+10H] |
| (7) MOV AX, [BP] | (8) MOV AX, VAL[BX+SI] |
| (9) MOV AX, VAL[BX+DI] | (10) MOV AX, [BP+DI] |

作业

6. ✓ 指出下列指令中哪些是错误的,错在什么地方。(提示:仅第(8)题是正确的。)

- | | |
|---------------------------|---------------------------|
| (1) MOV DL, AX | (2) MOV 8650H, AX |
| (3) MOV DS, 0200H | (4) MOV [BX], [1200H] |
| (5) MOV IP, 0FFH | (6) MOV [BX+SI+3], IP |
| (7) MOV AX, [BX+BP] | (8) MOV AL, ES:[BP] |
| (9) MOV DL, [SI+DI] | (10) MOV AX, OFFSET 0A20H |
| (11) MOV AL, OFFSET TABLE | (12) XCHG AL, 50H |
| (13) IN BL, 05H | (14) OUT AL, 0FFEh |

7. ✓ 已知当前 SS=1050H, SP=0100H, AX=4860H, BX=1287H, 试用示意图表示执行下列指令过程中, 堆栈中的内容和堆栈指针 SP 是怎样变化的。(参考例 3.29。)

```
PUSH AX
PUSH BX
POP BX
POP AX
```

作业

10. 已知 $AX=2508H$, $BX=0F36H$, $CX=0004H$, $DX=1864H$, 求下列每条指令执行后的结果是什么? 标志位 CF 等于什么?

(1) AND AH, CL

(2) OR BL, 30H

(3) NOT AX

(4) XOR CX, 0FFF0H

(5) TEST DH, 0FH

(6) CMP CX, 00H

(7) SHR DX, CL

(8) SAR AL, 1

(9) SHL BH, CL

(10) SAL AX, 1

(11) RCL BX, 1

(12) ROR DX, CL